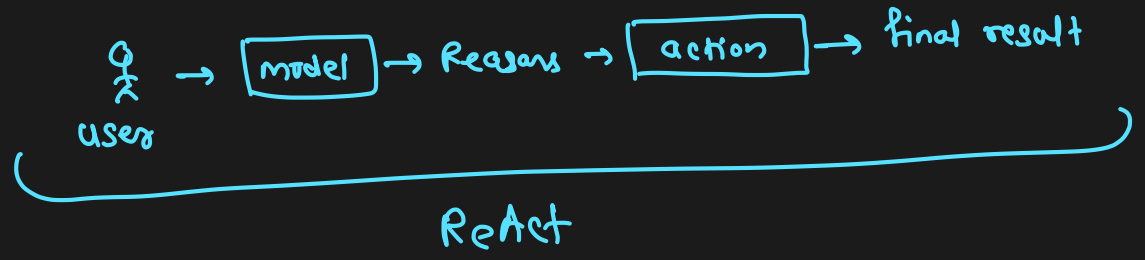
Reflection

Introduction

- Reflection is a design pattern where an agent **critiques** and **improves its own output** through iterative cycles before delivering a final answer
- Instead of accepting the first response, the agent generates, evaluates, and refines — mimicking how humans review their own work
- The name comes from the idea of "reflecting on your work" — the same way a writer drafts, reviews, edits, and re-reviews before publishing
- Reflection is a feedback loop in which an agent (or set of agents) produces an output, critiques that output against a goal or rubric, and uses the critique to produce an improved output — repeating until a stopping condition is met



Why does it exist?



- LLMs generate output **autoregressively and greedily** - each token is chosen based on prior tokens, with no internal checkpoint for correctness
- This causes predictable failure modes
  - **Hallucination:** Confident but false facts
  - **Edge-case blindness:** Misses null, empty, or boundary inputs
  - **Requirement drift:** Solves a related but wrong problem
  - **Reasoning errors:** Math, logic, or causal mistakes
  - **Style/format violations:** Wrong schema, tone, or structure
  - **Incomplete work:** Stops before finishing all steps
- A single pass has **no mechanism** to catch these. Reflection introduces one: a second look.
- LLMs are better at evaluating than generating
- Given a candidate answer, a model can often spot what's wrong even if it couldn't produce the right answer cold
- Reflection exploits this asymmetry

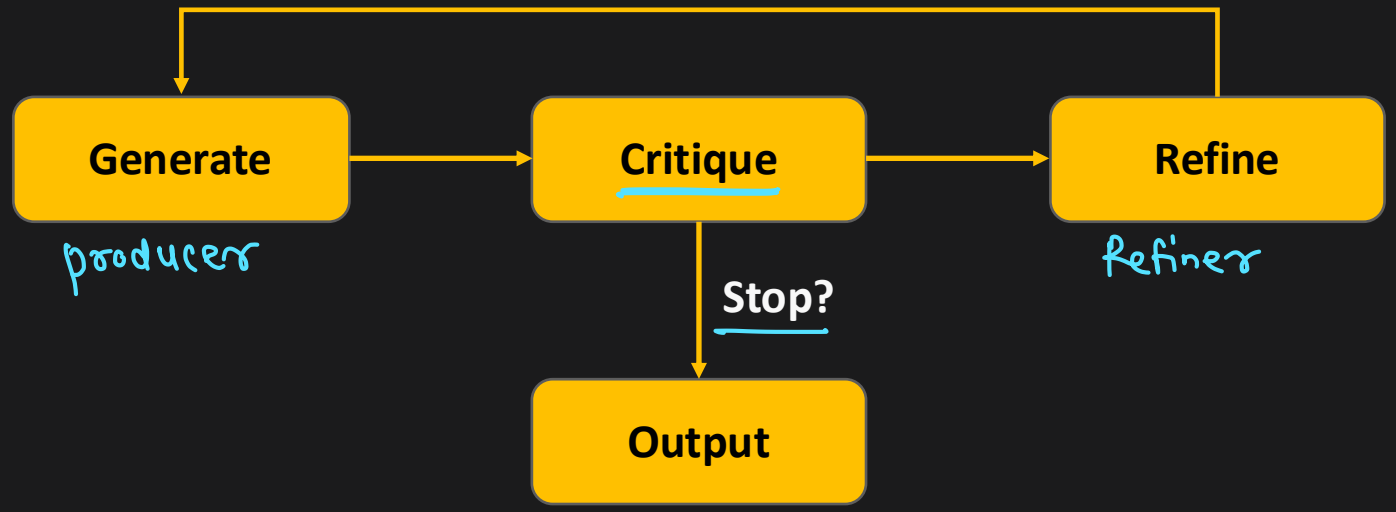
Key Components



- Producer
  - Generates the initial output
- Critic
  - Evaluates output against criteria; returns structured feedback
- Refiner
  - Rewrites output using critic feedback
- Stopping criteria
  - Max iterations + approval threshold
- Feedback schema
  - Structured output (severity, missing, suggestions)
- Ground truth
  - Optional tool/test results fed to the critic

The Fundamental Loop

- At its core, reflection is a **feedback control loop**



- **Three primitives**
  - **Generate** — produce a candidate output
  - **Critique** — evaluate it against criteria
  - **Refine** — produce an improved candidate
- **One decision point**
  - **Stop** — decide whether to accept, continue, or bail out

The Four Axes of Variation

- **Axis 1: Who critiques?**
  - **Self** — same agent, same prompt
  - **Self with role-switch** — same agent, different persona ("now act as a reviewer")
  - **Separate agent** — dedicated critic, ideally different model
  - **External tool** — tests, linters, validators
- **Axis 2: How is the critique shaped?**
  - **Free-form prose** — "here's what I think"
  - **Structured** — JSON/Pydantic with fields like severity, missing, suggestions
  - **Scored** — numeric rating against a rubric
  - **Binary** — pass/fail
  - **Grounded** — tied to external evidence (test output)
- **Axis 3: How is refinement scoped?**
  - **Full regeneration** — rewrite from scratch
  - **Surgical revision** — change only what the critique flagged
  - **Additive** — append missing pieces without touching existing ones
- **Axis 4: When does it stop?**
  - **Approval flag** — critic says "good enough"
  - **Severity threshold** — no issues above minor
  - **Max iterations** — hard cap
  - **Budget exhausted** — tokens, time, or cost

Architectural Variations

Self-Reflection (Single Agent)



- **Mechanism**
  - One agent, one conversation. The prompt instructs the model to critique its own output, then rewrite
- **Pros**
  - Simplest to implement
  - Cheapest (one agent, no handoff)
  - Preserves full context automatically
- **Cons**
  - **Shared blind spots** — if the model didn't know it was wrong, it won't know it's wrong on review
  - **Self-approval bias** — models tend to rate their own output favorably
  - **Weak signal** — critique is often vague ("looks good, maybe check edge cases")
- **Best for**
  - Low-stakes tasks, drafting, brainstorming, cost-sensitive applications.

Separate Critic Agent (Two Agents)



- **Mechanism**
  - A dedicated critic agent (often a different model or strongly different prompt) evaluates the producer's output
  - The producer revises based on critique
- **Pros**
  - **Breaks shared bias** — critic isn't anchored to producer's reasoning
  - **Stronger critique** — critic is prompted to be adversarial
  - **Clear separation** — one generates, one evaluates
- **Cons**
  - Higher cost (2× model calls per iteration)
  - Critic needs full context (task + output) to evaluate well
  - Handoff contract must be well-defined
- **Best for**
  - Code review, fact-checking, quality-critical content
- **Key design choice**
  - Use a **different model** for the critic when possible. Same-model critics still share training biases

Producer–Critic–Refiner (Three Agents)

- **Mechanism**
  - Splits evaluation from revision
  - Producer makes the first draft; Critic evaluates; Refiner applies feedback. Refiner and Critic alternate
- **Pros**
  - **Cleanest responsibilities** — each agent has one job
  - **Refiner can be scoped tightly** — "only fix what the critique lists"
  - **Reduces oscillation** — refiner doesn't re-litigate the whole design
- **Cons**
  - Most expensive (3 agents)
  - More orchestration complexity
  - Diminishing returns vs. Variation B for many tasks
- **Best for**
  - High-stakes generation, long-form content, complex code changes.

When to Use It



- **Use reflection when**
  - Output quality matters more than latency
  - The task has a **verifiable or rubric-able** success criterion
  - Errors are **costly** (production code, legal, medical, financial)
  - The model can **evaluate better than it can generate** (common for code)
  - You have **budget** for 2–3× inference cost
- **Don't use reflection when**
  - Task is simple lookup or classification
  - Latency is critical (real-time chat, autocomplete)
  - No clear quality signal exists
  - Output is cheap to fix downstream (drafts, brainstorms)
  - You can't afford the token cost